

ZERAVIA

TRAINING & INTERNSHIP PROGRAM

MINI PROJECT ASSIGNMENT

Machine Learning with Python

Student Name : Prerana Dey

Client Assignment

I have been assigned a practical machine learning task for a retail client that wants to use historical customer data to predict purchasing behaviour. I will work as a Junior Machine Learning Developer and build a complete beginner-level ML workflow from raw data preparation to model evaluation.

Client Scenario

The client has historical customer records containing demographic, engagement and purchasing information. Management wants to identify patterns in customer behaviour and build a model that can estimate whether a customer is likely to make a purchase, helping the business improve campaign targeting and customer engagement

Project Title

Customer Purchase Prediction & Behaviour Analysis

Contents

1	Project Setup & Problem Understanding	3
1.1	Business Problem Statement	3
1.2	ML Objective & Target Framing	3
2	Python Fundamentals	3
3	Data Loading & File Handling	3
4	NumPy & Exploratory Analysis	4
5	Pandas Data Cleaning	5
6	Distribution Inspection and correlation analysis	6
7	Data Preprocessing	10
8	Train-Test Split	10
9	Logistic Regression	11
10	Model Evaluation	11
11	Client Recommendations	14

Abstract

This project focuses on predicting customer churn using historical demographic, engagement, and purchasing data. A complete machine learning workflow was performed, including data cleaning, exploratory data analysis, missing-value imputation, outlier treatment, feature preprocessing, and model evaluation. Logistic Regression with balanced class weights was used for binary churn classification. The model achieved an accuracy of 80% and an ROC-AUC of 0.88, with a recall of 0.81 for churned customers. The analysis identified Tenure as an important factor associated with churn. Based on the results, targeted retention strategies such as early customer engagement, complaint resolution, recency-based reminders, and probability-based targeting are recommended.

1 Project Setup & Problem Understanding

1.1 Business Problem Statement

The retail client seeks to leverage historical customer data—encompassing demographics, engagement, and purchasing history—to understand purchase patterns and predict customer churn or purchase likelihood. High customer acquisition costs make retention and target-oriented marketing essential. By proactively identifying customers at risk of churning or those likely to buy, the business can deploy targeted engagement campaigns, offer tailored cashback incentives, and optimize promotional budgets.

1.2 ML Objective & Target Framing

- **Task Type:** Supervised Binary Classification.
- **Target Variable (y):** Churn $\in \{0, 1\}$, where 1 denotes a churned/high-risk customer and 0 denotes a retained/active customer.
- **Expected Output:** Predicted class $\hat{y} \in \{0, 1\}$ and continuous predicted probability $P(\text{Churn} = 1 \mid X)$.
- **Business Value:** Supports decision-making by prioritizing high-probability churn cases for loyalty interventions and segmenting customer lists to increase campaign ROI.

2 Python Fundamentals

Data loading, variable assignment, and column type selection executed using core Python features.

```
1 import pandas as pd
2 import numpy as np
```

Listing 1: Data Loading and Structural Selection

3 Data Loading & File Handling

Inspecting summary distribution and checking class balances.

```

1 raw_data=pd.read_csv("data_ecommerce_customer_churn.csv")
2 raw_data.head()
3 raw_data.shape
4 raw_data.columns
5 raw_data.dtypes
6 numeric_cols=['Tenure', 'WarehouseToHome', 'NumberOfDeviceRegistered', 'SatisfactionScore', 'NumberOfAddress', 'DaySinceLastOrder', 'CashbackAmount',
7 ]
8 numeric_data=raw_data[numeric_cols]
9 categorical_cols=raw_data.columns.difference(numeric_cols)
10 categorical_data=raw_data[categorical_cols]
11 print(categorical_data.head())
12 print(numeric_data.head())
13 numeric_data.describe()

```

Listing 2: Checking Summary Statistics

	Tenure	Warehouse	DeviceRegistered	SatisfactionScore	Address	LastOrder	CashbackAmt
Count	3747	3772	3941	3941	3941	3728	3941
Mean	10.0814	15.6506	3.6793	3.0883	4.2378	4.5317	176.7074
Std. Dev.	8.4989	8.4523	1.0139	1.3818	2.6267	3.6676	48.7918
Minimum	0.0000	5.0000	1.0000	1.0000	1.0000	0.0000	0.0000
25%	2.0000	9.0000	3.0000	2.0000	2.0000	2.0000	145.7000
Median	9.0000	14.0000	4.0000	3.0000	3.0000	3.0000	163.3400
75%	16.0000	21.0000	4.0000	4.0000	6.0000	7.0000	195.2500
Maximum	61.0000	127.0000	6.0000	5.0000	22.0000	46.0000	324.9900

Table 1: Summary Statistics of Numerical Variables

```

1 raw_data['Churn'].value_counts()
2 raw_data['Churn'].value_counts(normalize=True)
3 raw_data['Churn'].value_counts(normalize=True)*100

```

Listing 3: Churn Distribution Checks

Churn	Proportion (%)
0 (Not Churned)	82.897742
1 (Churned)	17.102258

Table 2: Proportion of Churned and Non-Churned Customers

4 NumPy & Exploratory Analysis

Visualizing target balances with Matplotlib.

```

1 import matplotlib.pyplot as plt
2 raw_data['Churn'].value_counts().plot(kind='bar')
3 plt.title('Churn Distribution')
4 plt.xlabel('Churn')
5 plt.ylabel('No of Customers')
6 plt.xticks([0,1],['Not Churned','Churned'],rotation=0)
7 plt.show()

```

Listing 4: Target Plotting

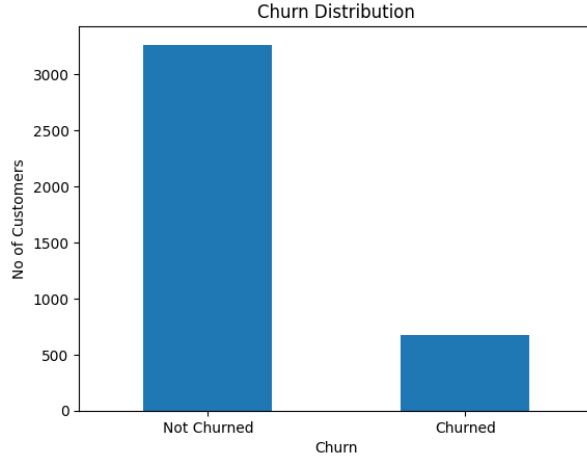


Figure 1: Churn distribution

Interpretation: The churn distribution shows that most customers are not churned, while a smaller proportion of customers have churned. This indicates that the dataset is imbalanced, with non-churned customers forming the majority class.

5 Pandas Data Cleaning

For a numerical feature X , the Interquartile Range (IQR) is defined as:

$$\text{IQR} = Q_3 - Q_1 \quad (1)$$

The bounds for flagging potential outliers are:

$$\text{Lower Bound} = Q_1 - 1.5 \times \text{IQR} \quad (2)$$

$$\text{Upper Bound} = Q_3 + 1.5 \times \text{IQR} \quad (3)$$

```

1 for col in numeric_cols:
2     q1=data_cleaned[col].quantile(0.25)
3     q3=data_cleaned[col].quantile(0.75)
4     iqr=q3-q1
5     lower_bound=q1-1.5*iqr
6     upper_bound=q3+1.5*iqr
7     outliers=data_cleaned[(data_cleaned[col]<lower_bound)|(data_cleaned[col]>
8         upper_bound)]
9     print(f'No of outliers in column{col}:{len(outliers)}')
```

Listing 5: Outlier Checking

Output :

No of outliers in columnTenure:4
 No of outliers in columnWarehouseToHome:1
 No of outliers in columnNumberOfDeviceRegistered:271
 No of outliers in columnSatisfactionScore:0
 No of outliers in columnNumberOfAddress:3
 No of outliers in columnDaySinceLastOrder:43
 No of outliers in columnCashbackAmount:316

```

1  for col in numeric_cols:
2      q1=data_cleaned[col].quantile(0.25)
3      q3=data_cleaned[col].quantile(0.75)
4      iqr=q3-q1
5      lower_bound=q1-1.5*iqr
6      upper_bound=q3+1.5*iqr
7      data_cleaned[col]=np.where(data_cleaned[col]<lower_bound,lower_bound,np.where
8      (data_cleaned[col]>upper_bound,upper_bound,data_cleaned[col]))
9  print(data_cleaned[numeric_cols].describe())
10
11 for col in numeric_cols:
12     q1=data_cleaned[col].quantile(0.25)
13     q3=data_cleaned[col].quantile(0.75)
14     iqr=q3-q1
15     lower_bound=q1-1.5*iqr
16     upper_bound=q3+1.5*iqr
17     outliers=data_cleaned[(data_cleaned[col]<lower_bound)|(data_cleaned[col]>
18     upper_bound)]
19     print(f'No of outliers in column{col}:{len(outliers)}')

```

Listing 6: Outlier Removing

No of outliers detected:0

6 Distribution Inspection and correlation analysis

Iterating through numerical columns to plot distributions and boxplots.

```

1  import matplotlib.pyplot as plt
2  import seaborn as sns
3  for col in numeric_cols:
4      plt.figure(figsize=(8,6))
5      sns.histplot(data=raw_data[col],kde=True)
6      plt.title(f'Distribution of {col}')
7      plt.xlabel(col)
8      plt.ylabel('Frequency')
9      plt.show()

```

Listing 7: Histogram Plot



Figure 2: Histogram

Interpretation: The histograms show that most numerical variables are not normally distributed. Tenure, WarehouseToHome, NumberOfAddress, and DaySinceLastOrder are positively skewed, with most observations concentrated at lower values. NumberOfDeviceRegistered and SatisfactionScore are concentrated around their middle values, while CashbackAmount is mainly concentrated between approximately 120 and 200. Overall, the variables show considerable variation in customer characteristics.

```

1 for col in numeric_cols:
2     sns.boxplot(data=raw_data[col])
3     plt.title(f'Boxplot of {col}')
4     plt.tight_layout()
5     plt.show()
6

```

Listing 8: Boxplot



Figure 3: Boxplot

Interpretation: The boxplots show the spread and possible outliers in the numerical variables. Tenure, WarehouseToHome, NumberOfAddress, DaySinceLastOrder, and CashbackAmount contain some extreme observations. NumberOfDeviceRegistered has a relatively narrow range, while SatisfactionScore is distributed between 1 and 5 with no clear extreme values. These observations were considered during the outlier treatment step.

```

1 for col in categorical_cols:
2     sns.countplot(data=raw_data, x=col, hue='Churn')
3     plt.title(f'Count plot of {col}')
4     plt.tight_layout()
5     plt.show()
6

```

Listing 9: Count Plot

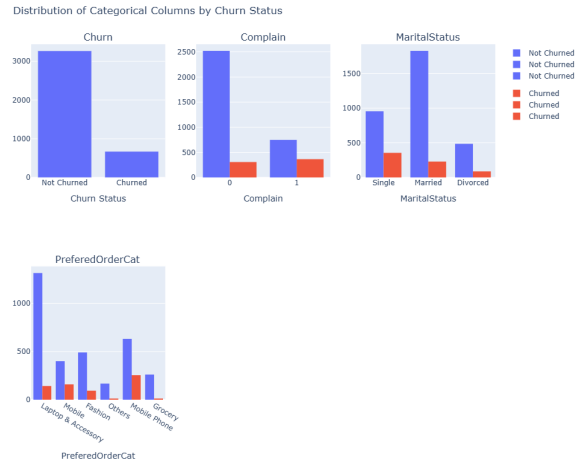


Figure 4: Countplot

Interpretation: The count plots show differences in customer churn across the categorical variables. For **Complain**, the number of churned customers is higher among customers who made a complaint. Among the marital-status groups, single customers have a higher number of churned customers compared with married and divorced customers. For **PreferredOrderCat**, the number of customers and churned customers varies across product categories, with Mobile Phone showing a relatively high number of churned customers.

```
1 corr=data_cleaned[numeric_cols+['Churn']].corr()
2 sns.heatmap(corr,annot=True,cmap='coolwarm')
3 plt.title('Correlation Heatmap')
4 plt.show()
```

Listing 10: Correlation checking

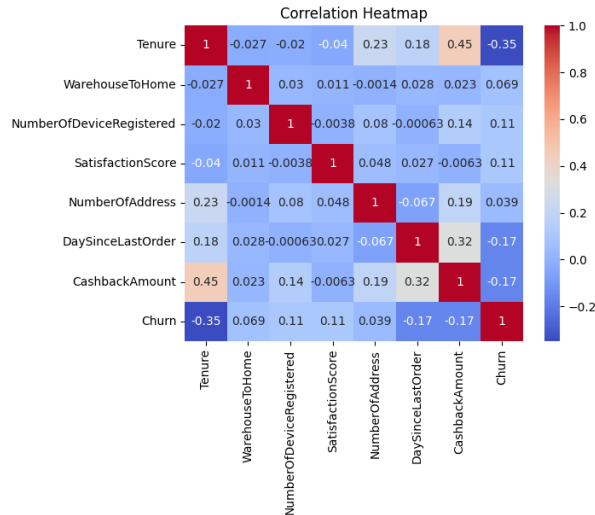


Figure 5: Correlation Heatmap

Interpretation: The heatmap shows that Tenure has the strongest correlation with Churn, with a moderate negative correlation of -0.35 . DaySinceLastOrder and CashbackAmount have weak negative correlations with Churn, while the remaining variables show very weak correlations. The correlations among the numerical predictors are generally low, indicating no major multicollinearity problem.

7 Data Preprocessing

Performing missing value imputation using median strategy.

```
1 data_cleaned=raw_data.copy()
2 data_cleaned.isnull().sum()
```

Listing 11: Missing value checking

Column	No of missing values
Tenure	194
WarehouseToHome	169
DaySinceLastOrder	213

Table 3: No of missing values

```
1 for col in numeric_cols:
2     median_val=data_cleaned[col].median()
3     data_cleaned[col]=data_cleaned[col].fillna(median_val)
4 print(data_cleaned.isnull().sum())
```

Listing 12: Missing value imputation

All these missing values have been taken care by imputing them with median values. Now the data has no missing value.

8 Train-Test Split

Dividing input variables (x) and target outcome (y) using stratified split logic.

```

1 x=data_cleaned.drop('Churn',axis=1)
2 y=data_cleaned['Churn']
3 print(x.shape)
4 print(y.shape)
5
6 from sklearn.model_selection import train_test_split
7 x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state
   =42,stratify=y)
8 print(x_train.shape)
9 print(x_test.shape)
10 print(y_train.shape)
11 print(y_test.shape)

```

Listing 13: Feature Target Separation and Stratified Splitting

9 Logistic Regression

Logistic regression bounds probabilities using the sigmoid function $\sigma(z)$:

$$P(y = 1 | X) = \sigma(\mathbf{w}^T X + b) = \frac{1}{1 + e^{-(\mathbf{w}^T X + b)}} \quad (4)$$

```

1 categorical_cols=['PreferredOrderCat','MaritalStatus','Complain']
2 numeric_cols=['Tenure','WarehouseToHome','NumberOfDeviceRegistered','
   SatisfactionScore','NumberOfAddress','DaySinceLastOrder','CashbackAmount']
3 from sklearn.preprocessing import StandardScaler,OneHotEncoder
4 from sklearn.compose import ColumnTransformer
5 preprocessor=ColumnTransformer(transformers=[('num',StandardScaler(),
   numeric_cols),('cat',OneHotEncoder(handle_unknown='ignore'),categorical_cols
   )])
6
7 from sklearn.linear_model import LogisticRegression
8 from sklearn.pipeline import Pipeline
9 model=Pipeline([('preprocessor',preprocessor),('classifier',LogisticRegression(
   max_iter=1000,class_weight="balanced"),),])
10 model.fit(x_train,y_train)
11 y_pred=model.predict(x_test)
12 print(y_pred)
13 y_prob=model.predict_proba(x_test)
14 print(y_prob)

```

Listing 14: Preprocessing Transformer and Model Pipeline Execution

10 Model Evaluation

Model performance evaluation formulas:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (5)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (6)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (7)$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (8)$$

```

1 from sklearn.metrics import accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report, roc_auc_score,
    RocCurveDisplay, roc_curve
2 accuracy=accuracy_score(y_test, y_pred)
3 precision=precision_score(y_test, y_pred)
4 recall=recall_score(y_test, y_pred)
5 f1=f1_score(y_test, y_pred)
6 roc_auc=roc_auc_score(y_test, y_prob[:, 1])
7 confusion_matrix=confusion_matrix(y_test, y_pred)
8 classification_report=classification_report(y_test, y_pred)
9 print('Accuracy:', accuracy)
10 print('Precision:', precision)
11 print('Recall:', recall)
12 print('F1 Score:', f1)
13 print('ROC_AUC:', roc_auc)
14 print('Confusion Matrix:')
15 print(confusion_matrix)
16 print('Classification_report:')
17 print(classification_report)
18 RocCurveDisplay.from_predictions(y_test, y_prob[:, 1])
19 plt.show()
20 train_acc=model.score(x_train, y_train)
21 test_acc=model.score(x_test, y_test)
22 print('Train Accuracy:', train_acc)
23 print('Test Accuracy:', test_acc)
24
25 from sklearn.metrics import ConfusionMatrixDisplay
26 ConfusionMatrixDisplay.from_predictions(y_test, y_pred, display_labels=['Not
    churned(0)', 'Churned(1)'])
27 plt.title('Confusion Matrix')
28 plt.show()

```

Listing 15: Evaluation Metrics

Churned status	Accuracy	Precision	Recall	F1 Score
0	0.80	0.95	0.79	0.87
1	0.80	0.45	0.81	0.58

Table 4: Classification Report

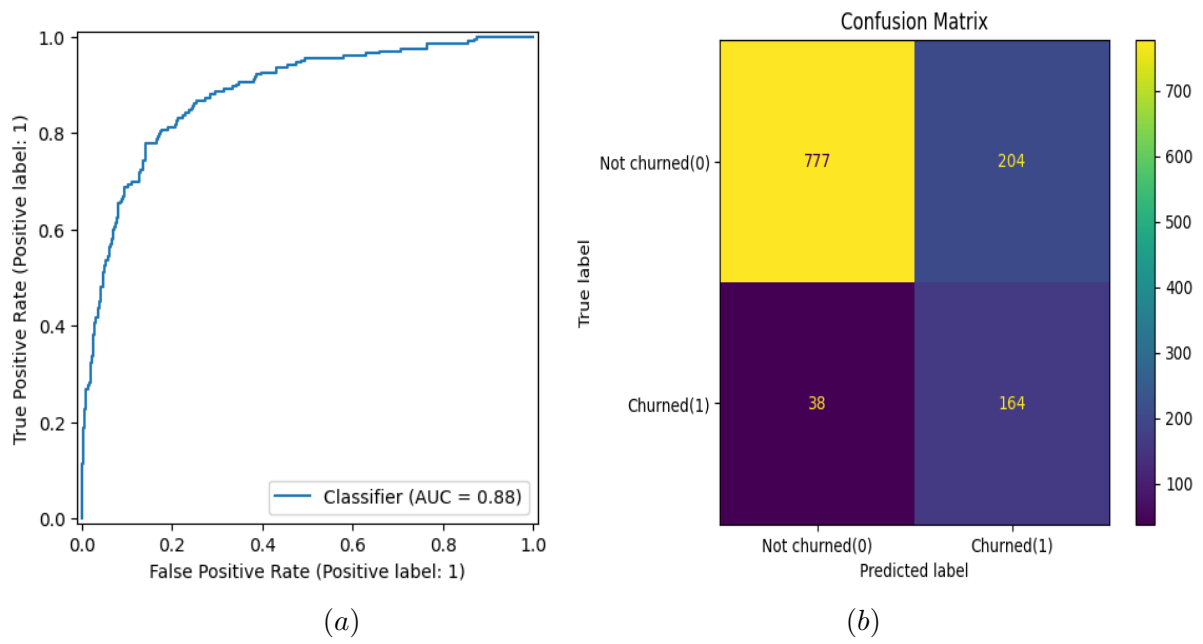


Figure 6: a) ROC-AUC Curve b) Confusion matrix

Interpretation:

a) The ROC curve shows that the Logistic Regression model has an AUC of 0.88. This indicates that the model has good ability to distinguish between churned and non-churned customers. The curve also shows a high true positive rate for relatively low false positive rates.

b) The confusion matrix shows that the model correctly classified 777 non-churned customers and 164 churned customers. It misclassified 204 non-churned customers as churned and 38 churned customers as non-churned. Overall, the model identifies most churned customers correctly, although some non-churned customers are incorrectly classified as churned.

11 Client Recommendations

Six strategic interventions based on data insights and model predictions:

1. **Early Tenure Onboarding:** Focus retention offers on new customers (Tenure < 3 months) through tailored onboarding sequences and exclusive welcome discounts.
2. **Rapid Complaint Resolution Workflow:** Flag customers who file complaints (Complain = 1) for priority resolution; negative service experiences strongly correlate with churn.
3. **Automated Recency Nudges:** Trigger automated SMS or email reminders when DaySinceLastOrder exceeds threshold limits (> 14 days).
4. **Targeted Cashback Incentives:** Optimize cashback allocation by offering dynamic rewards to high churn probability groups rather than universal distributions.
5. **Multi-Device Sync Engagement:** Enhance multi-device user experiences (e.g., app-to-web sync) to maintain engagement among users with multiple registered devices.
6. **Deploy Probability-Based Targeting:** Integrate model output probabilities ($P(\text{Churn}) > 0.65$) into CRM tools to trigger automated retention workflows before customers churn.